CONVERSOR DE ARQUIVOS

Documentação linha por linha do código

Cada linha dos arquivos ativos é numerada e explicada. Linhas em branco também são registradas como separação lógica.

Jose Diogo de Lima

Como usar este material

Leia a coluna esquerda como código real e a coluna direita como tradução para linguagem simples. A documentação segue o snapshot consolidado da arquitetura construída: Manifest, HTML, CSS, popup.js, service-worker.js, install.sh e converter_host.py.

Observação de fidelidade

O ambiente atual não conseguiu baixar novamente o commit remoto do GitHub. Por isso, este caderno usa o snapshot consolidado dos arquivos desenvolvidos nesta conversa. Se uma linha do seu projeto local foi alterada depois, compare o número e atualize a explicação correspondente.

1. manifest.json

Configuração que informa ao Chrome como carregar e autorizar a extensão.

1	`{`	Abre um bloco, objeto, lista ou agrupamento que será completado nas linhas seguintes.
2	`  "manifest_version": 3,`	Define Manifest V3, modelo atual usado pela extensão.
3	`  "name": "Conversor de Arquivos",`	Define uma propriedade e seu valor dentro do objeto JSON.
4	`  "version": "1.7.0",`	Define uma propriedade e seu valor dentro do objeto JSON.
5	`  "description": "Conversor local de áudio, vídeo, documentos, PDF, texto e imagens.",`	Define uma propriedade e seu valor dentro do objeto JSON.
6	`  "icons": {`	Define uma propriedade e seu valor dentro do objeto JSON.
7	`    "16": "assets/icons/icon16.png",`	Associa um tamanho de ícone ao arquivo PNG correspondente.
8	`    "48": "assets/icons/icon48.png",`	Associa um tamanho de ícone ao arquivo PNG correspondente.
9	`    "128": "assets/icons/icon128.png"`	Associa um tamanho de ícone ao arquivo PNG correspondente.
10	`  },`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
11	`  "action": {`	Define uma propriedade e seu valor dentro do objeto JSON.
12	`    "default_title": "Conversor de Arquivos",`	Define uma propriedade e seu valor dentro do objeto JSON.
13	`    "default_icon": {`	Define uma propriedade e seu valor dentro do objeto JSON.
14	`      "16": "assets/icons/icon16.png",`	Associa um tamanho de ícone ao arquivo PNG correspondente.
15	`      "48": "assets/icons/icon48.png",`	Associa um tamanho de ícone ao arquivo PNG correspondente.
16	`      "128": "assets/icons/icon128.png"`	Associa um tamanho de ícone ao arquivo PNG correspondente.
17	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
18	`  },`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
19	`  "background": {`	Define uma propriedade e seu valor dentro do objeto JSON.
20	`    "service_worker": "background/service-worker.js"`	Indica o arquivo JavaScript executado em segundo plano.
21	`  },`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
22	`  "permissions": [`	Define uma propriedade e seu valor dentro do objeto JSON.
23	`    "nativeMessaging",`	Autoriza a extensão a conversar com o programa local registrado.
24	`    "storage"`	Autoriza armazenamento local e de sessão.
25	`  ],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
26	`  "content_security_policy": {`	Inicia as regras de segurança de conteúdo da extensão.
27	`    "extension_pages": "default-src 'self'; script-src 'self'; object-src 'none'; style-src 'self'; img-src 'self'; base-uri 'none'; form-action 'none'"`	Define uma propriedade e seu valor dentro do objeto JSON.
28	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
29	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

2. popup/popup.html

Estrutura visual: botões, campos, mensagens e textos.

1	`<!DOCTYPE html>`	Declara que o documento usa HTML5.
2	`<html lang="pt-BR">`	Abre o documento e define português do Brasil como idioma.
3	`<head>`	Abre ou define um elemento visual da interface.
4	`  <meta charset="UTF-8">`	Define UTF-8 para exibir acentos corretamente.
5	`  <meta name="viewport" content="width=device-width, initial-scale=1.0">`	Configura o layout para acompanhar a largura da janela.
6	`  <title>Conversor de Arquivos</title>`	Abre ou define um elemento visual da interface.
7	`  <link rel="stylesheet" href="popup.css">`	Carrega o arquivo CSS da interface.
8	`  <script src="popup.js" defer></script>`	Carrega popup.js; defer espera o HTML ser analisado.
9	`</head>`	Fecha o elemento HTML aberto anteriormente.
10	`<body>`	Abre ou define um elemento visual da interface.
11	`  <main class="container">`	Abre ou define um elemento visual da interface.
12	`    <header class="header">`	Abre ou define um elemento visual da interface.
13	`      <div class="logo" aria-hidden="true">⇄</div>`	Começa oculto até o JavaScript remover o estado hidden.
14	`      <div>`	Abre ou define um elemento visual da interface.
15	`        <h1>Conversor de Arquivos</h1>`	Abre ou define um elemento visual da interface.
16	`        <p>Conversão local e segura</p>`	Abre ou define um elemento visual da interface.
17	`      </div>`	Fecha o elemento HTML aberto anteriormente.
18	`    </header>`	Fecha o elemento HTML aberto anteriormente.
19	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
20	`    <section class="card" aria-labelledby="converter-title">`	Abre ou define um elemento visual da interface.
21	`      <h2 id="converter-title">Converter arquivo</h2>`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
22	`      <button id="pick-button" class="button primary" type="button">`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
23	`        1. Escolher arquivo do computador`	Texto visível apresentado ao usuário dentro do elemento HTML.
24	`      </button>`	Fecha o elemento HTML aberto anteriormente.
25	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
26	`      <div id="selected-box" class="selected" hidden>`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
27	`        <span>Arquivo selecionado</span>`	Abre ou define um elemento visual da interface.
28	`        <strong id="selected-name"></strong>`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
29	`        <small id="selected-type"></small>`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
30	`        <button id="change-button" class="button secondary" type="button">`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
31	`          Trocar arquivo`	Texto visível apresentado ao usuário dentro do elemento HTML.
32	`        </button>`	Fecha o elemento HTML aberto anteriormente.
33	`      </div>`	Fecha o elemento HTML aberto anteriormente.
34	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
35	`      <div class="grid">`	Abre ou define um elemento visual da interface.
36	`        <div class="field">`	Abre ou define um elemento visual da interface.

37	`          <label for="format">2. Converter para</label>`	Abre ou define um elemento visual da interface.
38	`          <select id="format" disabled>`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
39	`            <option value="">Escolha o arquivo primeiro</option>`	Abre ou define um elemento visual da interface.
40	`          </select>`	Fecha o elemento HTML aberto anteriormente.
41	`        </div>`	Fecha o elemento HTML aberto anteriormente.
42	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
43	`        <div id="quality-field" class="field" hidden>`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
44	`          <label for="quality">Qualidade</label>`	Abre ou define um elemento visual da interface.
45	`          <select id="quality">`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
46	`            <option value="baixa">Baixa</option>`	Abre ou define um elemento visual da interface.
47	`            <option value="media" selected>Média</option>`	Abre ou define um elemento visual da interface.
48	`            <option value="alta">Alta</option>`	Abre ou define um elemento visual da interface.
49	`          </select>`	Fecha o elemento HTML aberto anteriormente.
50	`        </div>`	Fecha o elemento HTML aberto anteriormente.
51	`      </div>`	Fecha o elemento HTML aberto anteriormente.
52	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
53	`      <button id="convert-button" class="button success" type="button" disabled>`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
54	`        3. Converter e salvar`	Texto visível apresentado ao usuário dentro do elemento HTML.
55	`      </button>`	Fecha o elemento HTML aberto anteriormente.
56	`      <output id="status" class="status" aria-live="polite">`	Cria um elemento com ID usado pelo JavaScript para localizar e alterar o controle.
57	`        Escolha um arquivo para começar.`	Texto visível apresentado ao usuário dentro do elemento HTML.
58	`      </output>`	Fecha o elemento HTML aberto anteriormente.
59	`      <p class="hint">`	Abre ou define um elemento visual da interface.
60	`        O arquivo é processado localmente no computador.`	Texto visível apresentado ao usuário dentro do elemento HTML.
61	`      </p>`	Fecha o elemento HTML aberto anteriormente.
62	`    </section>`	Fecha o elemento HTML aberto anteriormente.
63	`    <footer>Versão 1.7.0</footer>`	Abre ou define um elemento visual da interface.
64	`  </main>`	Fecha o elemento HTML aberto anteriormente.
65	`</body>`	Fecha o elemento HTML aberto anteriormente.
66	`</html>`	Fecha o elemento HTML aberto anteriormente.

3. popup/popup.css

Aparência e estados visuais da janela.

1	`* {`	Inicia um conjunto de regras CSS para o seletor indicado.
2	`  box-sizing: border-box;`	Define a propriedade CSS box-sizing para controlar aparência ou layout.
3	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
4	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
5	`[hidden] {`	Inicia um conjunto de regras CSS para o seletor indicado.
6	`  display: none !important;`	Define a propriedade CSS display para controlar aparência ou layout.
7	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
8	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
9	`body {`	Inicia um conjunto de regras CSS para o seletor indicado.
10	`  margin: 0;`	Define a propriedade CSS margin para controlar aparência ou layout.
11	`  width: 380px;`	Define a propriedade CSS width para controlar aparência ou layout.
12	`  background: #0f172a;`	Define a propriedade CSS background para controlar aparência ou layout.
13	`  color: #f8fafc;`	Define a propriedade CSS color para controlar aparência ou layout.
14	`  font-family: "Segoe UI", Arial, sans-serif;`	Define a propriedade CSS font-family para controlar aparência ou layout.
15	`  font-size: 14px;`	Define a propriedade CSS font-size para controlar aparência ou layout.
16	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
17	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
18	`.container {`	Inicia um conjunto de regras CSS para o seletor indicado.
19	`  display: grid;`	Define a propriedade CSS display para controlar aparência ou layout.
20	`  gap: 14px;`	Define a propriedade CSS gap para controlar aparência ou layout.
21	`  padding: 16px;`	Define a propriedade CSS padding para controlar aparência ou layout.
22	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
23	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
24	`.header {`	Inicia um conjunto de regras CSS para o seletor indicado.
25	`  display: flex;`	Define a propriedade CSS display para controlar aparência ou layout.
26	`  align-items: center;`	Define a propriedade CSS align-items para controlar aparência ou layout.
27	`  gap: 12px;`	Define a propriedade CSS gap para controlar aparência ou layout.
28	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

29	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
30	.header h1,	Parte de um seletor CSS que identifica os elementos que receberão estilo.
31	.header p {	Inicia um conjunto de regras CSS para o seletor indicado.
32	margin: 0;	Define a propriedade CSS margin para controlar aparência ou layout.
33	}	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
34	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
35	.header h1 {	Inicia um conjunto de regras CSS para o seletor indicado.
36	font-size: 18px;	Define a propriedade CSS font-size para controlar aparência ou layout.
37	}	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
38	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
39	.header p {	Inicia um conjunto de regras CSS para o seletor indicado.
40	color: #94a3b8;	Define a propriedade CSS color para controlar aparência ou layout.
41	font-size: 12px;	Define a propriedade CSS font-size para controlar aparência ou layout.
42	}	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
43	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
44	.logo {	Inicia um conjunto de regras CSS para o seletor indicado.
45	display: grid;	Define a propriedade CSS display para controlar aparência ou layout.
46	place-items: center;	Define a propriedade CSS place-items para controlar aparência ou layout.
47	width: 42px;	Define a propriedade CSS width para controlar aparência ou layout.
48	height: 42px;	Define a propriedade CSS height para controlar aparência ou layout.
49	border-radius: 11px;	Define a propriedade CSS border-radius para controlar aparência ou layout.
50	background: #7c3aed;	Define a propriedade CSS background para controlar aparência ou layout.
51	color: #ffffff;	Define a propriedade CSS color para controlar aparência ou layout.
52	font-size: 23px;	Define a propriedade CSS font-size para controlar aparência ou layout.
53	font-weight: 700;	Define a propriedade CSS font-weight para controlar aparência ou layout.
54	}	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
55	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
56	.card {	Inicia um conjunto de regras CSS para o seletor indicado.
57	display: grid;	Define a propriedade CSS display para controlar aparência ou layout.
58	gap: 13px;	Define a propriedade CSS gap para controlar aparência ou layout.

59	`  padding: 14px;`	Define a propriedade CSS padding para controlar aparência ou layout.
60	`  background: #1e293b;`	Define a propriedade CSS background para controlar aparência ou layout.
61	`  border: 1px solid #334155;`	Define a propriedade CSS border para controlar aparência ou layout.
62	`  border-radius: 12px;`	Define a propriedade CSS border-radius para controlar aparência ou layout.
63	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
64	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
65	`.button {`	Inicia um conjunto de regras CSS para o seletor indicado.
66	`  width: 100%;`	Define a propriedade CSS width para controlar aparência ou layout.
67	`  padding: 10px;`	Define a propriedade CSS padding para controlar aparência ou layout.
68	`  border: 0;`	Define a propriedade CSS border para controlar aparência ou layout.
69	`  border-radius: 8px;`	Define a propriedade CSS border-radius para controlar aparência ou layout.
70	`  color: #ffffff;`	Define a propriedade CSS color para controlar aparência ou layout.
71	`  font-weight: 700;`	Define a propriedade CSS font-weight para controlar aparência ou layout.
72	`  cursor: pointer;`	Define a propriedade CSS cursor para controlar aparência ou layout.
73	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
74	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
75	`.button:disabled {`	Inicia um conjunto de regras CSS para o seletor indicado.
76	`  background: #475569 !important;`	Define a propriedade CSS background para controlar aparência ou layout.
77	`  color: #94a3b8;`	Define a propriedade CSS color para controlar aparência ou layout.
78	`  cursor: not-allowed;`	Define a propriedade CSS cursor para controlar aparência ou layout.
79	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
80	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
81	`.primary {`	Inicia um conjunto de regras CSS para o seletor indicado.
82	`  background: #2563eb;`	Define a propriedade CSS background para controlar aparência ou layout.
83	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
84	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
85	`.success {`	Inicia um conjunto de regras CSS para o seletor indicado.
86	`  background: #059669;`	Define a propriedade CSS background para controlar aparência ou layout.
87	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

88	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
89	.secondary {	Inicia um conjunto de regras CSS para o seletor indicado.
90	background: #334155;	Define a propriedade CSS background para controlar aparência ou layout.
91	border: 1px solid #475569;	Define a propriedade CSS border para controlar aparência ou layout.
92	}	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
93	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
94	.selected {	Inicia um conjunto de regras CSS para o seletor indicado.
95	display: grid;	Define a propriedade CSS display para controlar aparência ou layout.
96	gap: 6px;	Define a propriedade CSS gap para controlar aparência ou layout.
97	padding: 10px;	Define a propriedade CSS padding para controlar aparência ou layout.
98	background: #052e16;	Define a propriedade CSS background para controlar aparência ou layout.
99	border: 1px solid #22c55e;	Define a propriedade CSS border para controlar aparência ou layout.
100	border-radius: 9px;	Define a propriedade CSS border-radius para controlar aparência ou layout.
101	}	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
102	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
103	.grid {	Inicia um conjunto de regras CSS para o seletor indicado.
104	display: grid;	Define a propriedade CSS display para controlar aparência ou layout.
105	grid-template-columns: 1fr 1fr;	Define a propriedade CSS grid-template-columns para controlar aparência ou layout.
106	gap: 10px;	Define a propriedade CSS gap para controlar aparência ou layout.
107	}	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
108	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
109	.field {	Inicia um conjunto de regras CSS para o seletor indicado.
110	display: grid;	Define a propriedade CSS display para controlar aparência ou layout.
111	gap: 6px;	Define a propriedade CSS gap para controlar aparência ou layout.
112	}	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
113	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
114	select {	Inicia um conjunto de regras CSS para o seletor indicado.
115	width: 100%;	Define a propriedade CSS width para controlar aparência ou layout.
116	padding: 10px;	Define a propriedade CSS padding para controlar aparência ou layout.
117	border: 1px solid #475569;	Define a propriedade CSS border para controlar aparência ou layout.

118	`  border-radius: 8px;`	Define a propriedade CSS border-radius para controlar aparência ou layout.
119	`  background: #0f172a;`	Define a propriedade CSS background para controlar aparência ou layout.
120	`  color: #f8fafc;`	Define a propriedade CSS color para controlar aparência ou layout.
121	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
122	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
123	`select:disabled {`	Inicia um conjunto de regras CSS para o seletor indicado.
124	`  opacity: 0.55;`	Define a propriedade CSS opacity para controlar aparência ou layout.
125	`  cursor: not-allowed;`	Define a propriedade CSS cursor para controlar aparência ou layout.
126	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
127	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
128	`.status {`	Inicia um conjunto de regras CSS para o seletor indicado.
129	`  display: block;`	Define a propriedade CSS display para controlar aparência ou layout.
130	`  color: #94a3b8;`	Define a propriedade CSS color para controlar aparência ou layout.
131	`  font-size: 12px;`	Define a propriedade CSS font-size para controlar aparência ou layout.
132	`  line-height: 1.4;`	Define a propriedade CSS line-height para controlar aparência ou layout.
133	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
134	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
135	`.status.error {`	Inicia um conjunto de regras CSS para o seletor indicado.
136	`  color: #f87171;`	Define a propriedade CSS color para controlar aparência ou layout.
137	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
138	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
139	`.status.success {`	Inicia um conjunto de regras CSS para o seletor indicado.
140	`  color: #4ade80;`	Define a propriedade CSS color para controlar aparência ou layout.
141	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
142	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
143	`.hint {`	Inicia um conjunto de regras CSS para o seletor indicado.
144	`  margin: 0;`	Define a propriedade CSS margin para controlar aparência ou layout.
145	`  color: #94a3b8;`	Define a propriedade CSS color para controlar aparência ou layout.
146	`  font-size: 11px;`	Define a propriedade CSS font-size para controlar aparência ou layout.
147	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

148	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
149	`footer {`	Inicia um conjunto de regras CSS para o seletor indicado.
150	`  text-align: center;`	Define a propriedade CSS text-align para controlar aparência ou layout.
151	`  color: #64748b;`	Define a propriedade CSS color para controlar aparência ou layout.
152	`  font-size: 11px;`	Define a propriedade CSS font-size para controlar aparência ou layout.
153	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

4. popup/popup.js

Estado da interface e mensagens enviadas ao service worker.

1	`"use strict";`	Ativa modo estrito do JavaScript para reduzir comportamentos ambíguos.
2	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
3	`const pickButton = document.getElementById("pick-button");`	Cria uma referência ao elemento HTML com o ID informado.
4	`const changeButton = document.getElementById("change-button");`	Cria uma referência ao elemento HTML com o ID informado.
5	`const selectedBox = document.getElementById("selected-box");`	Cria uma referência ao elemento HTML com o ID informado.
6	`const selectedName = document.getElementById("selected-name");`	Cria uma referência ao elemento HTML com o ID informado.
7	`const selectedType = document.getElementById("selected-type");`	Cria uma referência ao elemento HTML com o ID informado.
8	`const formatSelect = document.getElementById("format");`	Cria uma referência ao elemento HTML com o ID informado.
9	`const qualityField = document.getElementById("quality-field");`	Cria uma referência ao elemento HTML com o ID informado.
10	`const qualitySelect = document.getElementById("quality");`	Cria uma referência ao elemento HTML com o ID informado.
11	`const convertButton = document.getElementById("convert-button");`	Cria uma referência ao elemento HTML com o ID informado.
12	`const statusText = document.getElementById("status");`	Cria uma referência ao elemento HTML com o ID informado.
13	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
14	`const STORAGE_KEY = "selectedConversionFile";`	Declara uma constante para reutilizar um valor sem reatribuição.
15	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
16	`const QUALITY_FORMATS = new Set([`	Cria uma coleção sem valores duplicados usada como lista branca.
17	`  "mp3", "m4a", "aac", "ogg", "opus", "ac3", "wma",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
18	`  "mp4", "mkv", "webm", "mov", "avi", "mpeg", "wmv", "flv", "gif_video",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
19	`  "jpg", "webp", "avif"`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
20	`]);`	Conclui uma instrução ou chamada de função.
21	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
22	`const FORMAT_LABELS = {`	Declara uma constante para reutilizar um valor sem reatribuição.
23	`  mp3: "MP3 (áudio)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
24	`  m4a: "M4A (áudio AAC)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
25	`  wav: "WAV (áudio sem perda)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
26	`  flac: "FLAC (áudio sem perda)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
27	`  mp4: "MP4 (vídeo)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
28	`  mkv: "MKV (vídeo)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
29	`  webm: "WEBM (vídeo)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.

30	`  jpg: "JPG (imagem)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
31	`  png: "PNG (imagem)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
32	`  webp: "WEBP (imagem)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
33	`  ocr_txt: "TXT por OCR",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
34	`  pdf: "PDF",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
35	`  docx: "DOCX (Word)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
36	`  odt: "ODT (Writer)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
37	`  txt: "TXT (texto)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
38	`  html: "HTML",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
39	`  xlsx: "XLSX (Excel)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
40	`  ods: "ODS (Calc)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
41	`  csv: "CSV",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
42	`  pptx: "PPTX (PowerPoint)",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
43	`  odp: "ODP (Impress)"`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
44	`};`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
45	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
46	`let selectionToken = null;`	Declara uma variável de estado cujo valor poderá mudar.
47	`let availableOutputs = [];`	Declara uma variável de estado cujo valor poderá mudar.
48	`let busy = false;`	Declara uma variável de estado cujo valor poderá mudar.
49	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
50	`function showStatus(message, type = "") {`	Declara uma função JavaScript reutilizável.
51	`  statusText.textContent = message;`	Define texto visível sem interpretar HTML.
52	`  statusText.className = type ? `status ${type}` : "status";`	Troca classes CSS para representar estado normal, erro ou sucesso.
53	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
54	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
55	`function setPlaceholder(text) {`	Declara uma função JavaScript reutilizável.
56	`  formatSelect.replaceChildren();`	Conclui uma instrução ou chamada de função.
57	`  const option = document.createElement("option");`	Declara uma constante para reutilizar um valor sem reatribuição.
58	`  option.value = "";`	Calcula ou armazena um valor usado pelo fluxo.
59	`  option.textContent = text;`	Define texto visível sem interpretar HTML.
60	`  formatSelect.appendChild(option);`	Conclui uma instrução ou chamada de função.
61	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

62	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
63	`function updateInterface() {`	Declara uma função JavaScript reutilizável.
64	`  const hasFile = typeof selectionToken === "string" && selectionToken.length > 0;`	Declara uma constante para reutilizar um valor sem reatribuição.
65	`  const selectedFormat = formatSelect.value;`	Declara uma constante para reutilizar um valor sem reatribuição.
66	`  const validFormat = availableOutputs.includes(selectedFormat);`	Declara uma constante para reutilizar um valor sem reatribuição.
67	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
68	`  pickButton.disabled = busy;`	Habilita ou desabilita o controle conforme a expressão booleana.
69	`  changeButton.disabled = busy;`	Habilita ou desabilita o controle conforme a expressão booleana.
70	`  formatSelect.disabled = busy || !hasFile || availableOutputs.length === 0;`	Habilita ou desabilita o controle conforme a expressão booleana.
71	`  qualityField.hidden = !validFormat || !QUALITY_FORMATS.has(selectedFormat);`	Mostra ou oculta o elemento conforme a expressão booleana.
72	`  qualitySelect.disabled = busy || qualityField.hidden;`	Habilita ou desabilita o controle conforme a expressão booleana.
73	`  convertButton.disabled = busy || !hasFile || !validFormat;`	Habilita ou desabilita o controle conforme a expressão booleana.
74	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
75	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
76	`function fillFormats(formats) {`	Declara uma função JavaScript reutilizável.
77	`  availableOutputs = Array.isArray(formats)`	Calcula ou armazena um valor usado pelo fluxo.
78	`    ? [...new Set(formats.filter(item => typeof item === "string" && item.length > 0))]`	Calcula ou armazena um valor usado pelo fluxo.
79	`    : [];`	Conclui uma instrução ou chamada de função.
80	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
81	`  formatSelect.replaceChildren();`	Conclui uma instrução ou chamada de função.
82	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
83	`  if (availableOutputs.length === 0) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
84	`    setPlaceholder("Nenhuma conversão disponível");`	Conclui uma instrução ou chamada de função.
85	`    updateInterface();`	Conclui uma instrução ou chamada de função.
86	`    return;`	Conclui uma instrução ou chamada de função.
87	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
88	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
89	`  const placeholder = document.createElement("option");`	Declara uma constante para reutilizar um valor sem reatribuição.
90	`  placeholder.value = "";`	Calcula ou armazena um valor usado pelo fluxo.
91	`  placeholder.textContent = "Escolha o formato";`	Define texto visível sem interpretar HTML.
92	`  formatSelect.appendChild(placeholder);`	Conclui uma instrução ou chamada de função.
93	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
94	`  for (const outputFormat of availableOutputs) {`	Percorre os itens da coleção indicada.
95	`    const option = document.createElement("option");`	Declara uma constante para reutilizar um valor sem reatribuição.
96	`    option.value = outputFormat;`	Calcula ou armazena um valor usado pelo fluxo.
97	`    option.textContent = FORMAT_LABELS[outputFormat] || outputFormat.toUpperCase();`	Define texto visível sem interpretar HTML.
98	`    formatSelect.appendChild(option);`	Conclui uma instrução ou chamada de função.

99	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
100	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
101	`  formatSelect.value = "";`	Calcula ou armazena um valor usado pelo fluxo.
102	`  updateInterface();`	Conclui uma instrução ou chamada de função.
103	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
104	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
105	`function showSelectedFile(selection) {`	Declara uma função JavaScript reutilizável.
106	`  selectionToken = selection.selection;`	Calcula ou armazena um valor usado pelo fluxo.
107	`  selectedName.textContent = selection.name || " arquivo";`	Define texto visível sem interpretar HTML.
108	`  selectedType.textContent = `Tipo detectado: ${selection.label || "Arquivo"}`;`	Define texto visível sem interpretar HTML.
109	`  selectedBox.hidden = false;`	Mostra ou oculta o elemento conforme a expressão booleana.
110	`  fillFormats(selection.outputs);`	Conclui uma instrução ou chamada de função.
111	`  showStatus("Arquivo carregado. Escolha o formato de saída.", "success");`	Conclui uma instrução ou chamada de função.
112	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
113	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
114	`function clearInterface() {`	Declara uma função JavaScript reutilizável.
115	`  selectionToken = null;`	Calcula ou armazena um valor usado pelo fluxo.
116	`  availableOutputs = [];`	Calcula ou armazena um valor usado pelo fluxo.
117	`  selectedBox.hidden = true;`	Mostra ou oculta o elemento conforme a expressão booleana.
118	`  selectedName.textContent = "";`	Define texto visível sem interpretar HTML.
119	`  selectedType.textContent = "";`	Define texto visível sem interpretar HTML.
120	`  setPlaceholder("Escolha o arquivo primeiro");`	Conclui uma instrução ou chamada de função.
121	`  updateInterface();`	Conclui uma instrução ou chamada de função.
122	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
123	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
124	`async function restoreSelection() {`	Declara uma função assíncrona capaz de usar await.
125	`  try {`	Inicia uma operação que pode falhar.
126	`    const response = await chrome.runtime.sendMessage({ action: "get-selection" });`	Declara uma constante para reutilizar um valor sem reatribuição.
127	`    if (response && response.success === true && response.selected === true) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
128	`      showSelectedFile(response);`	Conclui uma instrução ou chamada de função.
129	`      return;`	Conclui uma instrução ou chamada de função.
130	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
131	`    clearInterface();`	Conclui uma instrução ou chamada de função.
132	`    showStatus("Escolha um arquivo para começar.");`	Conclui uma instrução ou chamada de função.
133	`  } catch (error) {`	Abre um novo bloco de instruções.
134	`    clearInterface();`	Conclui uma instrução ou chamada de função.
135	`    showStatus("Não foi possível recuperar o arquivo selecionado.", "error");`	Conclui uma instrução ou chamada de função.

136	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
137	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
138	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
139	`async function chooseFile() {`	Declara uma função assíncrona capaz de usar await.
140	`  if (busy) return;`	Inicia uma condição; o bloco executa quando ela for verdadeira.
141	`  busy = true;`	Calcula ou armazena um valor usado pelo fluxo.
142	`  updateInterface();`	Conclui uma instrução ou chamada de função.
143	`  showStatus("Escolha o arquivo na janela que abriu...");`	Conclui uma instrução ou chamada de função.
144	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
145	`  try {`	Inicia uma operação que pode falhar.
146	`    const response = await chrome.runtime.sendMessage({ action: "pick-file" });`	Declara uma constante para reutilizar um valor sem reatribuição.
147	`    if (response && response.cancelled) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
148	`      showStatus(selectionToken ? "Seleção mantida." : "Nenhum arquivo selecionado.");`	Conclui uma instrução ou chamada de função.
149	`      return;`	Conclui uma instrução ou chamada de função.
150	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
151	`    if (!response || response.success !== true) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
152	`      throw new Error(response?.error || "Não foi possível selecionar o arquivo.");`	Conclui uma instrução ou chamada de função.
153	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
154	`    showSelectedFile(response);`	Conclui uma instrução ou chamada de função.
155	`  } catch (error) {`	Abre um novo bloco de instruções.
156	`    showStatus(error?.message || "Erro inesperado ao selecionar o arquivo.", "error");`	Conclui uma instrução ou chamada de função.
157	`  } finally {`	Abre um novo bloco de instruções.
158	`    busy = false;`	Calcula ou armazena um valor usado pelo fluxo.
159	`    updateInterface();`	Conclui uma instrução ou chamada de função.
160	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
161	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
162	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
163	`async function changeFile() {`	Declara uma função assíncrona capaz de usar await.
164	`  if (busy) return;`	Inicia uma condição; o bloco executa quando ela for verdadeira.
165	`  try {`	Inicia uma operação que pode falhar.
166	`    await chrome.runtime.sendMessage({ action: "clear-selection" });`	Envia uma mensagem da interface ao service worker.
167	`  } catch {`	Abre um novo bloco de instruções.
168	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
169	`  clearInterface();`	Conclui uma instrução ou chamada de função.
170	`  await chooseFile();`	Espera a Promise terminar antes de continuar esta função assíncrona.

171	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
172	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
173	`async function convertFile() {`	Declara uma função assíncrona capaz de usar await.
174	`  if (busy || !selectionToken) return;`	Inicia uma condição; o bloco executa quando ela for verdadeira.
175	`  const selectedFormat = formatSelect.value;`	Declara uma constante para reutilizar um valor sem reatribuição.
176	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
177	`  if (!availableOutputs.includes(selectedFormat)) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
178	`    showStatus("Escolha primeiro o formato de saída.", "error");`	Conclui uma instrução ou chamada de função.
179	`    return;`	Conclui uma instrução ou chamada de função.
180	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
181	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
182	`  busy = true;`	Calcula ou armazena um valor usado pelo fluxo.
183	`  updateInterface();`	Conclui uma instrução ou chamada de função.
184	`  showStatus("Escolha a pasta de destino...");`	Conclui uma instrução ou chamada de função.
185	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
186	`  try {`	Inicia uma operação que pode falhar.
187	`    const response = await chrome.runtime.sendMessage({`	Declara uma constante para reutilizar um valor sem reatribuição.
188	`      action: "convert-file",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
189	`      format: selectedFormat,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
190	`      quality: qualityField.hidden ? "media" : qualitySelect.value`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
191	`    });`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
192	`    if (response && response.cancelled) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
193	`      showStatus("Conversão cancelada. O arquivo continua selecionado.");`	Conclui uma instrução ou chamada de função.
194	`      return;`	Conclui uma instrução ou chamada de função.
195	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
196	`    if (!response || response.success !== true) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
197	`      throw new Error(response?.error || "Não foi possível converter o arquivo.");`	Conclui uma instrução ou chamada de função.
198	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
199	`    const outputName = response.output || "arquivo convertido";`	Declara uma constante para reutilizar um valor sem reatribuição.
200	`    clearInterface();`	Conclui uma instrução ou chamada de função.
201	`    showStatus(`Conversão concluída: ${outputName}`, "success");`	Conclui uma instrução ou chamada de função.
202	`  } catch (error) {`	Abre um novo bloco de instruções.
203	`    showStatus(error?.message || "Erro inesperado durante a conversão.", "error");`	Conclui uma instrução ou chamada de função.
204	`  } finally {`	Abre um novo bloco de instruções.
205	`    busy = false;`	Calcula ou armazena um valor usado pelo fluxo.

206	`    updateInterface();`	Conclui uma instrução ou chamada de função.
207	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
208	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
209	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
210	`pickButton.addEventListener("click", chooseFile);`	Registra uma função para responder ao evento indicado.
211	`changeButton.addEventListener("click", changeFile);`	Registra uma função para responder ao evento indicado.
212	`formatSelect.addEventListener("change", updateInterface);`	Registra uma função para responder ao evento indicado.
213	`qualitySelect.addEventListener("change", updateInterface);`	Registra uma função para responder ao evento indicado.
214	`convertButton.addEventListener("click", convertFile);`	Registra uma função para responder ao evento indicado.
215	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
216	`chrome.storage.onChanged.addListener((changes, areaName) => {`	Registra uma função para responder ao evento indicado.
217	`  if (areaName !== "local" || !changes[STORAGE_KEY] || busy) return;`	Inicia uma condição; o bloco executa quando ela for verdadeira.
218	`  const newSelection = changes[STORAGE_KEY].newValue;`	Declara uma constante para reutilizar um valor sem reatribuição.
219	`  if (newSelection && newSelection.selection) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
220	`    showSelectedFile(newSelection);`	Conclui uma instrução ou chamada de função.
221	`  } else {`	Abre um novo bloco de instruções.
222	`    clearInterface();`	Conclui uma instrução ou chamada de função.
223	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
224	`});`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
225	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
226	`setPlaceholder("Carregando...");`	Conclui uma instrução ou chamada de função.
227	`updateInterface();`	Conclui uma instrução ou chamada de função.
228	`restoreSelection();`	Conclui uma instrução ou chamada de função.

5. background/service-worker.js

Janela única, validação, storage e comunicação com o Python.

1	`"use strict";`	Ativa modo estrito do JavaScript para reduzir comportamentos ambíguos.
2	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
3	`const NATIVE_HOST_NAME = "com.mediadownloader.converter";`	Declara uma constante para reutilizar um valor sem reatribuição.
4	`const PAGE_URL = chrome.runtime.getURL("popup/popup.html");`	Declara uma constante para reutilizar um valor sem reatribuição.
5	`const STORAGE_KEY = "selectedConversionFile";`	Declara uma constante para reutilizar um valor sem reatribuição.
6	`const WINDOW_KEY = "converterWindowId";`	Declara uma constante para reutilizar um valor sem reatribuição.
7	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
8	`const ALLOWED_OUTPUT_FORMATS = new Set([`	Cria uma coleção sem valores duplicados usada como lista branca.
9	`  "mp3", "m4a", "aac", "ogg", "opus", "wav", "flac", "ac3", "aiff", "wma",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
10	`  "mp4", "mkv", "webm", "mov", "avi", "mpeg", "wmv", "flv", "gif_video",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
11	`  "jpg", "png", "webp", "bmp", "tiff", "gif", "avif", "ico", "ocr_txt",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
12	`  "pdf", "docx", "doc", "odt", "rtf", "txt", "html", "epub",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
13	`  "xlsx", "xls", "ods", "csv", "pptx", "ppt", "odp"`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
14	`]);`	Conclui uma instrução ou chamada de função.
15	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
16	`const ALLOWED_QUALITIES = new Set(["baixa", "media", "alta"]);`	Cria uma coleção sem valores duplicados usada como lista branca.
17	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
18	`chrome.action.onClicked.addListener(async () => {`	Registra uma função para responder ao evento indicado.
19	`  const stored = await chrome.storage.session.get(WINDOW_KEY);`	Declara uma constante para reutilizar um valor sem reatribuição.
20	`  const oldWindowId = stored[WINDOW_KEY];`	Declara uma constante para reutilizar um valor sem reatribuição.
21	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
22	`  if (typeof oldWindowId === "number") {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
23	`    try {`	Inicia uma operação que pode falhar.
24	`      await chrome.windows.update(oldWindowId, { focused: true });`	Atualiza a janela existente, normalmente trazendo-a para frente.
25	`      return;`	Conclui uma instrução ou chamada de função.
26	`    } catch {`	Abre um novo bloco de instruções.
27	`      await chrome.storage.session.remove(WINDOW_KEY);`	Espera a Promise terminar antes de continuar esta função assíncrona.
28	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
29	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
30	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.

31	`  const createdWindow = await chrome.windows.create({`	Declara uma constante para reutilizar um valor sem reatribuição.
32	`    url: PAGE_URL,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
33	`    type: "popup",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
34	`    width: 420,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
35	`    height: 660,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
36	`    focused: true`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
37	`  });`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
38	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
39	`  if (typeof createdWindow.id === "number") {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
40	`    await chrome.storage.session.set({ [WINDOW_KEY]: createdWindow.id });`	Espera a Promise terminar antes de continuar esta função assíncrona.
41	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
42	`});`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
43	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
44	`chrome.windows.onRemoved.addListener(async windowId => {`	Registra uma função para responder ao evento indicado.
45	`  const stored = await chrome.storage.session.get(WINDOW_KEY);`	Declara uma constante para reutilizar um valor sem reatribuição.
46	`  if (stored[WINDOW_KEY] === windowId) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
47	`    await chrome.storage.session.remove(WINDOW_KEY);`	Espera a Promise terminar antes de continuar esta função assíncrona.
48	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
49	`});`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
50	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
51	`function messageCameFromPage(sender) {`	Declara uma função JavaScript reutilizável.
52	`  return Boolean(`	Encerra a função e devolve o valor indicado.
53	`    sender &&`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
54	`    sender.id === chrome.runtime.id &&`	Calcula ou armazena um valor usado pelo fluxo.
55	`    typeof sender.url === "string" &&`	Calcula ou armazena um valor usado pelo fluxo.
56	`    sender.url.startsWith(PAGE_URL)`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
57	`  );`	Conclui uma instrução ou chamada de função.
58	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
59	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
60	`function isValidSelectionToken(token) {`	Declara uma função JavaScript reutilizável.
61	`  return typeof token === "string" && /^[a-f0-9]{32}$/.test(token);`	Encerra a função e devolve o valor indicado.
62	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

63	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
64	`function hasExactKeys(message, expectedKeys) {`	Declara uma função JavaScript reutilizável.
65	`  const actual = Object.keys(message).sort();`	Declara uma constante para reutilizar um valor sem reatribuição.
66	`  const expected = [...expectedKeys].sort();`	Declara uma constante para reutilizar um valor sem reatribuição.
67	`  return actual.length === expected.length &&`	Encerra a função e devolve o valor indicado.
68	`    actual.every((key, index) => key === expected[index]);`	Calcula ou armazena um valor usado pelo fluxo.
69	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
70	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
71	`function isValidMessage(message) {`	Declara uma função JavaScript reutilizável.
72	`  if (!message || typeof message !== "object" || Array.isArray(message)) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
73	`    return false;`	Encerra a função e devolve o valor indicado.
74	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
75	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
76	`  if (["pick-file", "get-selection", "clear-selection"].includes(message.action)) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
77	`    return hasExactKeys(message, ["action"]);`	Encerra a função e devolve o valor indicado.
78	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
79	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
80	`  if (message.action === "convert-file") {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
81	`    return hasExactKeys(message, ["action", "format", "quality"]) &&`	Encerra a função e devolve o valor indicado.
82	`      ALLOWED_OUTPUT_FORMATS.has(message.format) &&`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
83	`      ALLOWED_QUALITIES.has(message.quality);`	Conclui uma instrução ou chamada de função.
84	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
85	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
86	`  return false;`	Encerra a função e devolve o valor indicado.
87	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
88	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
89	`function friendlyNativeError(value) {`	Declara uma função JavaScript reutilizável.
90	`  const text = String(value || "");`	Declara uma constante para reutilizar um valor sem reatribuição.
91	`  if (text.includes("Specified native messaging host not found")) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
92	`    return "Conversor local não encontrado.";`	Encerra a função e devolve o valor indicado.
93	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
94	`  if (text.includes("forbidden")) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
95	`    return "Esta extensão não está autorizada a usar o conversor local.";`	Encerra a função e devolve o valor indicado.
96	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

97	`  if (text.includes("Native host has exited")) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
98	`    return "O conversor local foi encerrado inesperadamente.";`	Encerra a função e devolve o valor indicado.
99	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
100	`  return text || "Não foi possível acessar o conversor local.";`	Encerra a função e devolve o valor indicado.
101	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
102	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
103	`function callNativeHost(payload) {`	Declara uma função JavaScript reutilizável.
104	`  return new Promise(resolve => {`	Encerra a função e devolve o valor indicado.
105	`    let port = null;`	Declara uma variável de estado cujo valor poderá mudar.
106	`    let finished = false;`	Declara uma variável de estado cujo valor poderá mudar.
107	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
108	`    function finish(response) {`	Declara uma função JavaScript reutilizável.
109	`      if (finished) return;`	Inicia uma condição; o bloco executa quando ela for verdadeira.
110	`      finished = true;`	Calcula ou armazena um valor usado pelo fluxo.
111	`      resolve(response);`	Conclui uma instrução ou chamada de função.
112	`      try {`	Inicia uma operação que pode falhar.
113	`        port?.disconnect();`	Conclui uma instrução ou chamada de função.
114	`      } catch {`	Abre um novo bloco de instruções.
115	`      }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
116	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
117	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
118	`    try {`	Inicia uma operação que pode falhar.
119	`      port = chrome.runtime.connectNative(NATIVE_HOST_NAME);`	Abre a conexão com o host Native Messaging.
120	`    } catch (error) {`	Abre um novo bloco de instruções.
121	`      finish({ success: false, error: friendlyNativeError(error?.message) });`	Conclui uma instrução ou chamada de função.
122	`      return;`	Conclui uma instrução ou chamada de função.
123	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
124	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
125	`    port.onMessage.addListener(finish);`	Registra uma função para responder ao evento indicado.
126	`    port.onDisconnect.addListener(() => {`	Registra uma função para responder ao evento indicado.
127	`      if (!finished) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
128	`        finish({`	Abre um novo bloco de instruções.
129	`          success: false,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
130	`          error: friendlyNativeError(chrome.runtime.lastError?.message)`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
131	`        });`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
132	`      }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

133	`    });`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
134	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
135	`    try {`	Inicia uma operação que pode falhar.
136	`      port.postMessage(payload);`	Envia o payload pela conexão Native Messaging.
137	`    } catch (error) {`	Abre um novo bloco de instruções.
138	`      finish({ success: false, error: friendlyNativeError(error?.message) });`	Conclui uma instrução ou chamada de função.
139	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
140	`  });`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
141	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
142	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
143	`async function saveSelection(selectionData) {`	Declara uma função assíncrona capaz de usar await.
144	`  await chrome.storage.local.set({ [STORAGE_KEY]: selectionData });`	Salva a seleção no armazenamento local da extensão.
145	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
146	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
147	`async function getSavedSelection() {`	Declara uma função assíncrona capaz de usar await.
148	`  const result = await chrome.storage.local.get(STORAGE_KEY);`	Declara uma constante para reutilizar um valor sem reatribuição.
149	`  const selection = result[STORAGE_KEY];`	Declara uma constante para reutilizar um valor sem reatribuição.
150	`  if (!selection || !isValidSelectionToken(selection.selection)) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
151	`    return null;`	Encerra a função e devolve o valor indicado.
152	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
153	`  return selection;`	Encerra a função e devolve o valor indicado.
154	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
155	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
156	`async function clearSavedSelection() {`	Declara uma função assíncrona capaz de usar await.
157	`  await chrome.storage.local.remove(STORAGE_KEY);`	Apaga a seleção do armazenamento.
158	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
159	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
160	`function sanitizeSelection(response) {`	Declara uma função JavaScript reutilizável.
161	`  if (!response || response.success !== true) return null;`	Inicia uma condição; o bloco executa quando ela for verdadeira.
162	`  if (!isValidSelectionToken(response.selection)) return null;`	Inicia uma condição; o bloco executa quando ela for verdadeira.
163	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
164	`  const outputs = Array.isArray(response.outputs)`	Declara uma constante para reutilizar um valor sem reatribuição.
165	`    ? response.outputs.filter(format => ALLOWED_OUTPUT_FORMATS.has(format))`	Calcula ou armazena um valor usado pelo fluxo.
166	`    : [];`	Conclui uma instrução ou chamada de função.

167	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
168	`  if (outputs.length === 0) return null;`	Inicia uma condição; o bloco executa quando ela for verdadeira.
169	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
170	`  return {`	Encerra a função e devolve o valor indicado.
171	`    selection: response.selection,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
172	`    name: String(response.name || "arquivo").slice(0, 255),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
173	`    label: String(response.label || "Arquivo").slice(0, 50),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
174	`    extension: String(response.extension || "").slice(0, 20),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
175	`    outputs,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
176	`    savedAt: Date.now()`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
177	`  };`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
178	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
179	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
180	`async function pickFile() {`	Declara uma função assíncrona capaz de usar await.
181	`  const nativeResponse = await callNativeHost({ action: "pick-file" });`	Declara uma constante para reutilizar um valor sem reatribuição.
182	`  if (nativeResponse?.cancelled) return { success: false, cancelled: true };`	Inicia uma condição; o bloco executa quando ela for verdadeira.
183	`  if (!nativeResponse || nativeResponse.success !== true) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
184	`    return { success: false, error: nativeResponse?.error || "Falha ao selecionar." };`	Encerra a função e devolve o valor indicado.
185	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
186	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
187	`  const selectionData = sanitizeSelection(nativeResponse);`	Declara uma constante para reutilizar um valor sem reatribuição.
188	`  if (!selectionData) return { success: false, error: "Dados inválidos." };`	Inicia uma condição; o bloco executa quando ela for verdadeira.
189	`  await saveSelection(selectionData);`	Espera a Promise terminar antes de continuar esta função assíncrona.
190	`  return { success: true, ...selectionData };`	Encerra a função e devolve o valor indicado.
191	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
192	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
193	`async function getSelection() {`	Declara uma função assíncrona capaz de usar await.
194	`  const selection = await getSavedSelection();`	Declara uma constante para reutilizar um valor sem reatribuição.
195	`  return selection`	Encerra a função e devolve o valor indicado.
196	`    ? { success: true, selected: true, ...selection }`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
197	`    : { success: true, selected: false };`	Conclui uma instrução ou chamada de função.
198	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

199	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
200	`async function convertFile(message) {`	Declara uma função assíncrona capaz de usar await.
201	`  const selection = await getSavedSelection();`	Declara uma constante para reutilizar um valor sem reatribuição.
202	`  if (!selection) return { success: false, error: "Nenhum arquivo está selecionado." };`	Inicia uma condição; o bloco executa quando ela for verdadeira.
203	`  if (!selection.outputs.includes(message.format)) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
204	`    return { success: false, error: "Formato incompatível." };`	Encerra a função e devolve o valor indicado.
205	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
206	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
207	`  const nativeResponse = await callNativeHost({`	Declara uma constante para reutilizar um valor sem reatribuição.
208	`    action: "convert-file",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
209	`    selection: selection.selection,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
210	`    format: message.format,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
211	`    quality: message.quality`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
212	`  });`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
213	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
214	`  if (nativeResponse?.cancelled) return { success: false, cancelled: true };`	Inicia uma condição; o bloco executa quando ela for verdadeira.
215	`  if (!nativeResponse || nativeResponse.success !== true) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
216	`    return { success: false, error: nativeResponse?.error || "Falha na conversão." };`	Encerra a função e devolve o valor indicado.
217	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
218	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
219	`  await clearSavedSelection();`	Espera a Promise terminar antes de continuar esta função assíncrona.
220	`  return { success: true, output: nativeResponse.output };`	Encerra a função e devolve o valor indicado.
221	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
222	(linha em branco)	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
223	`async function processRequest(message) {`	Declara uma função assíncrona capaz de usar await.
224	`  if (message.action === "pick-file") return pickFile();`	Inicia uma condição; o bloco executa quando ela for verdadeira.
225	`  if (message.action === "get-selection") return getSelection();`	Inicia uma condição; o bloco executa quando ela for verdadeira.
226	`  if (message.action === "clear-selection") {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
227	`    await clearSavedSelection();`	Espera a Promise terminar antes de continuar esta função assíncrona.
228	`    return { success: true };`	Encerra a função e devolve o valor indicado.
229	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
230	`  if (message.action === "convert-file") return convertFile(message);`	Inicia uma condição; o bloco executa quando ela for verdadeira.

231	`  return { success: false, error: "Ação inválida." };`	Encerra a função e devolve o valor indicado.
232	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
233	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
234	`chrome.runtime.onMessage.addListener((message, sender, sendResponse) => {`	Registra uma função para responder ao evento indicado.
235	`  if (!messageCameFromPage(sender)) return false;`	Inicia uma condição; o bloco executa quando ela for verdadeira.
236	`  if (!isValidMessage(message)) {`	Inicia uma condição; o bloco executa quando ela for verdadeira.
237	`    sendResponse({ success: false, error: "Pedido inválido." });`	Conclui uma instrução ou chamada de função.
238	`    return false;`	Encerra a função e devolve o valor indicado.
239	`  }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
240	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
241	`  processRequest(message)`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
242	`    .then(sendResponse)`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
243	`    .catch(error => sendResponse({`	Calcula ou armazena um valor usado pelo fluxo.
244	`      success: false,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
245	`      error: error?.message || "Erro inesperado."`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
246	`    }));`	Conclui uma instrução ou chamada de função.
247	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
248	`  return true;`	Encerra a função e devolve o valor indicado.
249	`});`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.

6. native-host/install.sh

Registro do host Native Messaging no Google Chrome.

1	`#!/usr/bin/env bash`	Shebang: informa ao Linux qual interpretador deve executar o arquivo.
2	`set -euo pipefail`	Ativa modo estrito: interrompe em erro, variável ausente ou falha de pipeline.
3	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
4	`HOST_NAME="com.mediadownloader.converter"`	Define uma variável do script para reutilizar um valor.
5	`EXTENSION_ID="${1:-}"`	Define uma variável do script para reutilizar um valor.
6	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
7	`if [[ ! "$EXTENSION_ID" =~ ^[a-p]{32}$ ]]; then`	Inicia uma condição do shell.
8	`  echo "Uso: bash native-host/install.sh ID_DA_EXTENSAO"`	Exibe uma orientação ou confirmação no terminal.
9	`  exit 1`	Encerra o instalador com código de erro.
10	`fi`	Encerra a condição iniciada por if.
11	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
12	`SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"`	Define uma variável do script para reutilizar um valor.
13	`HOST_PATH="$SCRIPT_DIR/converter_host.py"`	Define uma variável do script para reutilizar um valor.
14	`TARGET_DIR="$HOME/.config/google-chrome/NativeMessagingHosts"`	Define uma variável do script para reutilizar um valor.
15	`TARGET_FILE="$TARGET_DIR/$HOST_NAME.json"`	Define uma variável do script para reutilizar um valor.
16	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
17	`chmod u+x "$HOST_PATH"`	Altera permissões do arquivo indicado.
18	`mkdir -p "$TARGET_DIR"`	Cria a pasta de destino se ainda não existir.
19	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
20	`cat > "$TARGET_FILE" <<JSON`	Inicia a gravação do manifesto Native Messaging.
21	`{`	Abre um bloco, objeto, lista ou agrupamento que será completado nas linhas seguintes.
22	`  "name": "$HOST_NAME",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
23	`  "description": "Conversor local de arquivos",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
24	`  "path": "$HOST_PATH",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
25	`  "type": "stdio",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
26	`  "allowed_origins": [`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
27	`    "chrome-extension://$EXTENSION_ID/"`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
28	`  ]`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
29	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
30	`JSON`	Finaliza o bloco de texto JSON gravado pelo heredoc.
31	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.

32	`chmod 600 "$TARGET_FILE"`	Altera permissões do arquivo indicado.
33	`echo "Host instalado em: $TARGET_FILE"`	Exibe uma orientação ou confirmação no terminal.

7. native-host/converter_host.py

Seleção, detecção, tokens e execução das conversões.

1	`#!/usr/bin/python3`	Shebang: informa ao Linux qual interpretador deve executar o arquivo.
2	`# -*- coding: utf-8 -*-`	Declara a codificação UTF-8 do arquivo Python.
3	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
4	`import html`	Importa um módulo da biblioteca padrão necessário ao programa.
5	`import json`	Importa um módulo da biblioteca padrão necessário ao programa.
6	`import os`	Importa um módulo da biblioteca padrão necessário ao programa.
7	`import re`	Importa um módulo da biblioteca padrão necessário ao programa.
8	`import secrets`	Importa um módulo da biblioteca padrão necessário ao programa.
9	`import shutil`	Importa um módulo da biblioteca padrão necessário ao programa.
10	`import struct`	Importa um módulo da biblioteca padrão necessário ao programa.
11	`import subprocess`	Importa um módulo da biblioteca padrão necessário ao programa.
12	`import sys`	Importa um módulo da biblioteca padrão necessário ao programa.
13	`import tempfile`	Importa um módulo da biblioteca padrão necessário ao programa.
14	`import time`	Importa um módulo da biblioteca padrão necessário ao programa.
15	`from pathlib import Path`	Importa uma classe ou função específica de outro módulo.
16	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
17	`HOME = Path.home().resolve()`	Calcula ou armazena um valor que será usado pelas próximas linhas.
18	`TOKENS = HOME / "MediaConverter" / "selections"`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
19	`MAX_MESSAGE = 64 * 1024`	Calcula ou armazena um valor que será usado pelas próximas linhas.
20	`TOKEN_TTL = 60 * 60`	Calcula ou armazena um valor que será usado pelas próximas linhas.
21	`DIALOG_TIMEOUT = 10 * 60`	Calcula ou armazena um valor que será usado pelas próximas linhas.
22	`CONVERT_TIMEOUT = 2 * 60 * 60`	Calcula ou armazena um valor que será usado pelas próximas linhas.
23	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
24	`FFMPEG = shutil.which("ffmpeg")`	Procura o executável no PATH e guarda o caminho ou None.
25	`FFPROBE = shutil.which("ffprobe")`	Procura o executável no PATH e guarda o caminho ou None.
26	`SOFFICE = shutil.which("soffice") or shutil.which("libreoffice")`	Procura o executável no PATH e guarda o caminho ou None.
27	`PDFTOTEXT = shutil.which("pdftotext")`	Procura o executável no PATH e guarda o caminho ou None.
28	`PDFTOPPM = shutil.which("pdftoppm")`	Procura o executável no PATH e guarda o caminho ou None.
29	`MAGICK = shutil.which("magick") or shutil.which("convert")`	Procura o executável no PATH e guarda o caminho ou None.

30	`TESSERACT = shutil.which("tesseract")`	Procura o executável no PATH e guarda o caminho ou None.
31	`ZENITY = shutil.which("zenity")`	Procura o executável no PATH e guarda o caminho ou None.
32	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
33	`CATEGORIES = {`	Calcula ou armazena um valor que será usado pelas próximas linhas.
34	`    "pdf": ({"pdf"}, ["txt", "png", "jpg"]),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
35	`    "image": (`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
36	`        {"jpg", "jpeg", "png", "webp", "bmp", "gif", "tif", "tiff", "avif", "ico"},`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
37	`        ["jpg", "png", "webp", "bmp", "tiff", "gif", "avif", "ico", "pdf", "ocr_txt"],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
38	`    ),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
39	`    "text": ({"txt", "md", "log"}, ["pdf", "docx", "odt", "rtf", "html", "txt"]),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
40	`    "document": (`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
41	`        {"doc", "docx", "odt", "rtf", "wps", "html", "epub"},`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
42	`        ["pdf", "docx", "doc", "odt", "rtf", "txt", "html", "epub"],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
43	`    ),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
44	`    "spreadsheet": ({"xls", "xlsx", "ods", "csv", "tsv"}, ["pdf", "xlsx", "xls", "ods", "csv"]),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
45	`    "presentation": ({"ppt", "pptx", "odp"}, ["pdf", "pptx", "ppt", "odp"]),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
46	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
47	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
48	`LABELS = {`	Calcula ou armazena um valor que será usado pelas próximas linhas.
49	`    "audio": "Áudio",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
50	`    "video": "Vídeo",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
51	`    "pdf": "PDF",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
52	`    "image": "Imagem",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
53	`    "text": "Texto",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
54	`    "document": "Documento",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
55	`    "spreadsheet": "Planilha",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
56	`    "presentation": "Apresentação",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
57	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
58	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.

59	`MEDIA_OUTPUTS = {`	Calcula ou armazena um valor que será usado pelas próximas linhas.
60	`    "audio": ["mp3", "m4a", "aac", "ogg", "opus", "wav", "flac", "ac3"],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
61	`    "video": ["mp4", "mkv", "webm", "mov", "avi", "mpeg", "mp3", "m4a", "wav", "flac"],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
62	`}`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
63	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
64	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
65	`def ok(**values):`	Declara uma função Python reutilizável.
66	`    return {"success": True, **values}`	Encerra a função e devolve o valor indicado ao chamador.
67	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
68	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
69	`def fail(message):`	Declara uma função Python reutilizável.
70	`    return {"success": False, "error": str(message)}`	Encerra a função e devolve o valor indicado ao chamador.
71	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
72	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
73	`def cancelled():`	Declara uma função Python reutilizável.
74	`    return {"success": False, "cancelled": True}`	Encerra a função e devolve o valor indicado ao chamador.
75	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
76	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
77	`def read_message():`	Declara uma função Python reutilizável.
78	`    header = sys.stdin.buffer.read(4)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
79	`    if len(header) < 4:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
80	`        return None`	Encerra a função e devolve o valor indicado ao chamador.
81	`    length = struct.unpack("@I", header)[0]`	Converte o cabeçalho binário recebido em número inteiro.
82	`    if length <= 0 or length > MAX_MESSAGE:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
83	`        raise ValueError("Mensagem inválida")`	Interrompe o fluxo atual lançando uma exceção.
84	`    data = sys.stdin.buffer.read(length)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
85	`    if len(data) != length:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
86	`        raise ValueError("Mensagem incompleta")`	Interrompe o fluxo atual lançando uma exceção.
87	`    return json.loads(data.decode("utf-8"))`	Encerra a função e devolve o valor indicado ao chamador.
88	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
89	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
90	`def send_message(message):`	Declara uma função Python reutilizável.
91	`    data = json.dumps(message, ensure_ascii=False).encode("utf-8")`	Serializa um objeto Python para texto JSON.

92	`    sys.stdout.buffer.write(struct.pack("@I", len(data)))`	Converte o tamanho da mensagem em cabeçalho binário de quatro bytes.
93	`    sys.stdout.buffer.write(data)`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
94	`    sys.stdout.buffer.flush()`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
95	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
96	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
97	`def run_dialog(arguments):`	Declara uma função Python reutilizável.
98	`    try:`	Inicia um bloco que pode gerar exceção.
99	`        result = subprocess.run(`	Executa um programa externo com argumentos controlados.
100	`            arguments,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
101	`            stdin=subprocess.DEVNULL,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
102	`            capture_output=True,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
103	`            timeout=DIALOG_TIMEOUT,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
104	`            check=False,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
105	`        )`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
106	`        return result.returncode, result.stdout.decode("utf-8", errors="replace").strip()`	Encerra a função e devolve o valor indicado ao chamador.
107	`    except (OSError, subprocess.TimeoutExpired):`	Trata a exceção indicada sem encerrar abruptamente o host.
108	`        return None, ""`	Encerra a função e devolve o valor indicado ao chamador.
109	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
110	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
111	`def choose_file():`	Declara uma função Python reutilizável.
112	`    if not ZENITY:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
113	`        return None, fail("Zenity não encontrado.")`	Encerra a função e devolve o valor indicado ao chamador.
114	`    code, raw = run_dialog([`	Calcula ou armazena um valor que será usado pelas próximas linhas.
115	`        ZENITY,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
116	`        "--file-selection",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
117	`        "--title=Escolha um arquivo para converter",`	Calcula ou armazena um valor que será usado pelas próximas linhas.
118	`        f"--filename={HOME}/",`	Calcula ou armazena um valor que será usado pelas próximas linhas.
119	`    ])`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
120	`    if code is None:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
121	`        return None, fail("Não foi possível abrir o seletor.")`	Encerra a função e devolve o valor indicado ao chamador.
122	`    if code != 0:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.

123	`        return None, cancelled()`	Encerra a função e devolve o valor indicado ao chamador.
124	`    try:`	Inicia um bloco que pode gerar exceção.
125	`        path = Path(raw).resolve(strict=True)`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
126	`    except (OSError, RuntimeError):`	Trata a exceção indicada sem encerrar abruptamente o host.
127	`        return None, fail("Arquivo inválido.")`	Encerra a função e devolve o valor indicado ao chamador.
128	`    if not path.is_file():`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
129	`        return None, fail("Escolha um arquivo comum.")`	Encerra a função e devolve o valor indicado ao chamador.
130	`    return path, None`	Encerra a função e devolve o valor indicado ao chamador.
131	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
132	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
133	`def choose_output_dir():`	Declara uma função Python reutilizável.
134	`    if not ZENITY:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
135	`        return None, fail("Zenity não encontrado.")`	Encerra a função e devolve o valor indicado ao chamador.
136	`    default = HOME / "Downloads"`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
137	`    code, raw = run_dialog([`	Calcula ou armazena um valor que será usado pelas próximas linhas.
138	`        ZENITY,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
139	`        "--file-selection",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
140	`        "--directory",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
141	`        "--title=Escolha onde salvar",`	Calcula ou armazena um valor que será usado pelas próximas linhas.
142	`        f"--filename={default}/",`	Calcula ou armazena um valor que será usado pelas próximas linhas.
143	`    ])`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
144	`    if code != 0:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
145	`        return None, cancelled()`	Encerra a função e devolve o valor indicado ao chamador.
146	`    try:`	Inicia um bloco que pode gerar exceção.
147	`        folder = Path(raw).resolve(strict=True)`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
148	`    except (OSError, RuntimeError):`	Trata a exceção indicada sem encerrar abruptamente o host.
149	`        return None, fail("Pasta inválida.")`	Encerra a função e devolve o valor indicado ao chamador.
150	`    if not folder.is_dir() or not os.access(folder, os.W_OK | os.X_OK):`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
151	`        return None, fail("Sem permissão para salvar.")`	Encerra a função e devolve o valor indicado ao chamador.
152	`    return folder, None`	Encerra a função e devolve o valor indicado ao chamador.
153	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
154	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
155	`def detect_media(path):`	Declara uma função Python reutilizável.
156	`    if not FFPROBE:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.

157	`        return None`	Encerra a função e devolve o valor indicado ao chamador.
158	`    try:`	Inicia um bloco que pode gerar exceção.
159	`        result = subprocess.run([`	Executa um programa externo com argumentos controlados.
160	`            FFPROBE,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
161	`            "-v", "error",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
162	`            "-show_entries", "stream=codec_type",`	Calcula ou armazena um valor que será usado pelas próximas linhas.
163	`            "-of", "json",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
164	`            str(path),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
165	`        ], capture_output=True, timeout=30, check=False)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
166	`        data = json.loads(result.stdout or b"{}")`	Converte texto JSON em objeto Python.
167	`        types = {item.get("codec_type") for item in data.get("streams", [])}`	Calcula ou armazena um valor que será usado pelas próximas linhas.
168	`        if "video" in types:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
169	`            return "video"`	Encerra a função e devolve o valor indicado ao chamador.
170	`        if "audio" in types:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
171	`            return "audio"`	Encerra a função e devolve o valor indicado ao chamador.
172	`    except Exception:`	Trata a exceção indicada sem encerrar abruptamente o host.
173	`        return None`	Encerra a função e devolve o valor indicado ao chamador.
174	`    return None`	Encerra a função e devolve o valor indicado ao chamador.
175	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
176	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
177	`def inspect_file(path):`	Declara uma função Python reutilizável.
178	`    extension = path.suffix.lower().lstrip(".")`	Calcula ou armazena um valor que será usado pelas próximas linhas.
179	`    for category, (extensions, outputs) in CATEGORIES.items():`	Inicia um laço que percorre uma coleção.
180	`        if extension in extensions:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
181	`            return category, list(outputs)`	Encerra a função e devolve o valor indicado ao chamador.
182	`    media = detect_media(path)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
183	`    if media:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
184	`        return media, list(MEDIA_OUTPUTS[media])`	Encerra a função e devolve o valor indicado ao chamador.
185	`    return None, []`	Encerra a função e devolve o valor indicado ao chamador.
186	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
187	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
188	`def cleanup_tokens():`	Declara uma função Python reutilizável.
189	`    TOKENS.mkdir(parents=True, exist_ok=True, mode=0o700)`	Calcula ou armazena um valor que será usado pelas próximas linhas.

190	`    now = time.time()`	Calcula ou armazena um valor que será usado pelas próximas linhas.
191	`    for token_file in TOKENS.glob("*.json"):`	Inicia um laço que percorre uma coleção.
192	`        try:`	Inicia um bloco que pode gerar exceção.
193	`            if now - token_file.stat().st_mtime > TOKEN_TTL:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
194	`                token_file.unlink(missing_ok=True)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
195	`        except OSError:`	Trata a exceção indicada sem encerrar abruptamente o host.
196	`            pass`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
197	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
198	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
199	`def save_token(path):`	Declara uma função Python reutilizável.
200	`    cleanup_tokens()`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
201	`    token = secrets.token_hex(16)`	Gera um token aleatório criptograficamente seguro.
202	`    token_file = TOKENS / f"{token}.json"`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
203	`    token_file.write_text(json.dumps({"path": str(path)}), encoding="utf-8")`	Serializa um objeto Python para texto JSON.
204	`    token_file.chmod(0o600)`	Restringe as permissões do arquivo temporário.
205	`    return token`	Encerra a função e devolve o valor indicado ao chamador.
206	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
207	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
208	`def load_token(token, consume=False):`	Declara uma função Python reutilizável.
209	`    cleanup_tokens()`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
210	`    if not isinstance(token, str) or not re.fullmatch(r"[a-f0-9]{32}", token):`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
211	`        return None`	Encerra a função e devolve o valor indicado ao chamador.
212	`    token_file = TOKENS / f"{token}.json"`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
213	`    try:`	Inicia um bloco que pode gerar exceção.
214	`        data = json.loads(token_file.read_text(encoding="utf-8"))`	Converte texto JSON em objeto Python.
215	`        path = Path(data["path"]).resolve(strict=True)`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
216	`    except Exception:`	Trata a exceção indicada sem encerrar abruptamente o host.
217	`        return None`	Encerra a função e devolve o valor indicado ao chamador.
218	`    if not path.is_file():`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
219	`        return None`	Encerra a função e devolve o valor indicado ao chamador.
220	`    if consume:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
221	`        token_file.unlink(missing_ok=True)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
222	`    return path`	Encerra a função e devolve o valor indicado ao chamador.

223	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
224	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
225	`def output_path(source, extension, folder):`	Declara uma função Python reutilizável.
226	`    stem = re.sub(r"[^\w\s().-]", "_", source.stem)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
227	`    stem = stem.strip(" .")[:100] or "arquivo"`	Calcula ou armazena um valor que será usado pelas próximas linhas.
228	`    output = folder / f"{stem}.{extension}"`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
229	`    number = 1`	Calcula ou armazena um valor que será usado pelas próximas linhas.
230	`    while output.exists() or output.is_symlink():`	Repete o bloco enquanto a condição for verdadeira.
231	`        output = folder / f"{stem} ({number}).{extension}"`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
232	`        number += 1`	Calcula ou armazena um valor que será usado pelas próximas linhas.
233	`    return output`	Encerra a função e devolve o valor indicado ao chamador.
234	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
235	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
236	`def run_command(command, output):`	Declara uma função Python reutilizável.
237	`    try:`	Inicia um bloco que pode gerar exceção.
238	`        result = subprocess.run(`	Executa um programa externo com argumentos controlados.
239	`            command,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
240	`            stdin=subprocess.DEVNULL,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
241	`            capture_output=True,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
242	`            timeout=CONVERT_TIMEOUT,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
243	`            check=False,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
244	`        )`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
245	`    except subprocess.TimeoutExpired:`	Trata a exceção indicada sem encerrar abruptamente o host.
246	`        output.unlink(missing_ok=True)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
247	`        return fail("Tempo limite excedido.")`	Encerra a função e devolve o valor indicado ao chamador.
248	`    if result.returncode != 0 or not output.exists():`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
249	`        output.unlink(missing_ok=True)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
250	`        details = result.stderr.decode("utf-8", errors="replace").strip()[-350:]`	Calcula ou armazena um valor que será usado pelas próximas linhas.
251	`        return fail(details or "Falha na conversão.")`	Encerra a função e devolve o valor indicado ao chamador.
252	`    return ok(output=output.name)`	Encerra a função e devolve o valor indicado ao chamador.
253	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
254	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.

255	`def convert_media(source, extension, quality, output):`	Declara uma função Python reutilizável.
256	`    if not FFMPEG:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
257	`        return fail("FFmpeg não encontrado.")`	Encerra a função e devolve o valor indicado ao chamador.
258	`    bitrate = {"baixa": "128k", "media": "192k", "alta": "320k"}.get(quality, "192k")`	Calcula ou armazena um valor que será usado pelas próximas linhas.
259	`    audio = {`	Calcula ou armazena um valor que será usado pelas próximas linhas.
260	`        "mp3": ["-vn", "-c:a", "libmp3lame", "-b:a", bitrate],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
261	`        "m4a": ["-vn", "-c:a", "aac", "-b:a", bitrate],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
262	`        "wav": ["-vn", "-c:a", "pcm_s16le"],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
263	`        "flac": ["-vn", "-c:a", "flac"],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
264	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
265	`    video = {`	Calcula ou armazena um valor que será usado pelas próximas linhas.
266	`        "mp4": ["-c:v", "libx264", "-c:a", "aac"],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
267	`        "mkv": ["-c:v", "libx264", "-c:a", "aac"],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
268	`        "webm": ["-c:v", "libvpx-vp9", "-c:a", "libopus"],`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
269	`    }`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
270	`    arguments = audio.get(extension) or video.get(extension)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
271	`    if not arguments:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
272	`        return fail("Formato de mídia inválido.")`	Encerra a função e devolve o valor indicado ao chamador.
273	`    return run_command([`	Encerra a função e devolve o valor indicado ao chamador.
274	`        FFMPEG,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
275	`        "-y",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
276	`        "-hide_banner",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
277	`        "-nostdin",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
278	`        "-loglevel", "error",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
279	`        "-i", str(source),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
280	`        *arguments,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
281	`        str(output),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
282	`    ], output)`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
283	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.

284	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
285	`def convert_image(source, extension, output):`	Declara uma função Python reutilizável.
286	`    if extension == "ocr_txt":`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
287	`        if not TESSERACT:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
288	`            return fail("Tesseract não encontrado.")`	Encerra a função e devolve o valor indicado ao chamador.
289	`        base = output.with_suffix("")`	Calcula ou armazena um valor que será usado pelas próximas linhas.
290	`        result = subprocess.run([`	Executa um programa externo com argumentos controlados.
291	`            TESSERACT,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
292	`            str(source),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
293	`            str(base),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
294	`            "-l", "por+eng",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
295	`        ], capture_output=True, timeout=CONVERT_TIMEOUT, check=False)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
296	`        return ok(output=output.name) if result.returncode == 0 and output.exists() else fail("Falha no OCR.")`	Encerra a função e devolve o valor indicado ao chamador.
297	`    if not MAGICK:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
298	`        return fail("ImageMagick não encontrado.")`	Encerra a função e devolve o valor indicado ao chamador.
299	`    return run_command([MAGICK, str(source), str(output)], output)`	Encerra a função e devolve o valor indicado ao chamador.
300	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
301	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
302	`def convert_pdf(source, extension, output):`	Declara uma função Python reutilizável.
303	`    if extension == "txt":`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
304	`        if not PDFTOTEXT:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
305	`            return fail("pdftotext não encontrado.")`	Encerra a função e devolve o valor indicado ao chamador.
306	`        return run_command([`	Encerra a função e devolve o valor indicado ao chamador.
307	`            PDFTOTEXT,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
308	`            "-layout",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
309	`            "-enc", "UTF-8",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
310	`            str(source),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
311	`            str(output),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
312	`        ], output)`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
313	`    if extension in {"png", "jpg"}:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.

314	`        if not PDFTOPPM:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
315	`            return fail("pdftoppm não encontrado.")`	Encerra a função e devolve o valor indicado ao chamador.
316	`        temp = Path(tempfile.mkdtemp(prefix="pdf-img-"))`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
317	`        try:`	Inicia um bloco que pode gerar exceção.
318	`            prefix = temp / "pagina"`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
319	`            option = "-png" if extension == "png" else "-jpeg"`	Calcula ou armazena um valor que será usado pelas próximas linhas.
320	`            subprocess.run([`	Executa um programa externo com argumentos controlados.
321	`                PDFTOPPM,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
322	`                "-f", "1",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
323	`                "-l", "1",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
324	`                "-singlefile",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
325	`                "-r", "150",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
326	`                option,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
327	`                str(source),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
328	`                str(prefix),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
329	`            ], capture_output=True, timeout=CONVERT_TIMEOUT, check=False)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
330	`            candidate = next(temp.glob("pagina.*"), None)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
331	`            if not candidate:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
332	`                return fail("Falha ao converter PDF.")`	Encerra a função e devolve o valor indicado ao chamador.
333	`            shutil.move(str(candidate), str(output))`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
334	`            return ok(output=output.name)`	Encerra a função e devolve o valor indicado ao chamador.
335	`        finally:`	Executa limpeza independentemente de sucesso ou erro.
336	`            shutil.rmtree(temp, ignore_errors=True)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
337	`    return fail("Conversão de PDF não suportada.")`	Encerra a função e devolve o valor indicado ao chamador.
338	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
339	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
340	`def convert_text(source, extension, output):`	Declara uma função Python reutilizável.
341	`    if extension == "txt":`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
342	`        shutil.copy2(source, output)`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
343	`        return ok(output=output.name)`	Encerra a função e devolve o valor indicado ao chamador.
344	`    if extension == "html":`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.

345	`        text = source.read_text(encoding="utf-8", errors="replace")`	Calcula ou armazena um valor que será usado pelas próximas linhas.
346	`        output.write_text(`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
347	`            "<!DOCTYPE html><html><body><pre>" + html.escape(text) + "</pre></body></html>",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
348	`            encoding="utf-8",`	Calcula ou armazena um valor que será usado pelas próximas linhas.
349	`        )`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
350	`        return ok(output=output.name)`	Encerra a função e devolve o valor indicado ao chamador.
351	`    return convert_office(source, extension, output)`	Encerra a função e devolve o valor indicado ao chamador.
352	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
353	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
354	`def convert_office(source, extension, output):`	Declara uma função Python reutilizável.
355	`    if not SOFFICE:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
356	`        return fail("LibreOffice não encontrado.")`	Encerra a função e devolve o valor indicado ao chamador.
357	`    temp = Path(tempfile.mkdtemp(prefix="office-"))`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
358	`    profile = Path(tempfile.mkdtemp(prefix="office-profile-"))`	Cria ou combina caminhos usando pathlib, evitando concatenação manual.
359	`    try:`	Inicia um bloco que pode gerar exceção.
360	`        result = subprocess.run([`	Executa um programa externo com argumentos controlados.
361	`            SOFFICE,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
362	`            f"-env:UserInstallation={profile.as_uri()}",`	Calcula ou armazena um valor que será usado pelas próximas linhas.
363	`            "--headless",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
364	`            "--norestore",`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
365	`            "--convert-to", extension,`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
366	`            "--outdir", str(temp),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
367	`            str(source),`	Adiciona um item ou propriedade à estrutura que continua nas linhas seguintes.
368	`        ], capture_output=True, timeout=CONVERT_TIMEOUT, check=False)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
369	`        candidate = next(temp.glob(f"*.{extension}"), None)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
370	`        if result.returncode != 0 or not candidate:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
371	`            return fail("LibreOffice não conseguiu converter.")`	Encerra a função e devolve o valor indicado ao chamador.
372	`        shutil.move(str(candidate), str(output))`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
373	`        return ok(output=output.name)`	Encerra a função e devolve o valor indicado ao chamador.
374	`    finally:`	Executa limpeza independentemente de sucesso ou erro.
375	`        shutil.rmtree(temp, ignore_errors=True)`	Calcula ou armazena um valor que será usado pelas próximas linhas.

376	`        shutil.rmtree(profile, ignore_errors=True)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
377	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
378	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
379	`def pick_file():`	Declara uma função Python reutilizável.
380	`    source, problem = choose_file()`	Calcula ou armazena um valor que será usado pelas próximas linhas.
381	`    if problem:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
382	`        return problem`	Encerra a função e devolve o valor indicado ao chamador.
383	`    category, outputs = inspect_file(source)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
384	`    if not category:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
385	`        return fail("Tipo não reconhecido.")`	Encerra a função e devolve o valor indicado ao chamador.
386	`    return ok(`	Encerra a função e devolve o valor indicado ao chamador.
387	`        selection=save_token(source),`	Calcula ou armazena um valor que será usado pelas próximas linhas.
388	`        name=source.name,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
389	`        extension=source.suffix.lower().lstrip("."),`	Calcula ou armazena um valor que será usado pelas próximas linhas.
390	`        label=LABELS.get(category, "Arquivo"),`	Calcula ou armazena um valor que será usado pelas próximas linhas.
391	`        outputs=outputs,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
392	`        size=source.stat().st_size,`	Calcula ou armazena um valor que será usado pelas próximas linhas.
393	`    )`	Fecha o bloco, objeto, lista, chamada ou declaração aberto anteriormente.
394	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
395	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
396	`def convert_file(message):`	Declara uma função Python reutilizável.
397	`    token = message.get("selection")`	Calcula ou armazena um valor que será usado pelas próximas linhas.
398	`    extension = message.get("format")`	Calcula ou armazena um valor que será usado pelas próximas linhas.
399	`    quality = message.get("quality", "media")`	Calcula ou armazena um valor que será usado pelas próximas linhas.
400	`    source = load_token(token)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
401	`    if source is None:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
402	`        return fail("Seleção inválida ou expirada.")`	Encerra a função e devolve o valor indicado ao chamador.
403	`    category, outputs = inspect_file(source)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
404	`    if not category or extension not in outputs:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
405	`        return fail("Formato incompatível.")`	Encerra a função e devolve o valor indicado ao chamador.
406	`    folder, problem = choose_output_dir()`	Calcula ou armazena um valor que será usado pelas próximas linhas.

407	`    if problem:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
408	`        return problem`	Encerra a função e devolve o valor indicado ao chamador.
409	`    final_extension = "txt" if extension == "ocr_txt" else extension`	Calcula ou armazena um valor que será usado pelas próximas linhas.
410	`    output = output_path(source, final_extension, folder)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
411	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
412	`    if category in {"audio", "video"}:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
413	`        result = convert_media(source, extension, quality, output)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
414	`    elif category == "image":`	Testa uma condição alternativa quando as anteriores falharam.
415	`        result = convert_image(source, extension, output)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
416	`    elif category == "pdf":`	Testa uma condição alternativa quando as anteriores falharam.
417	`        result = convert_pdf(source, extension, output)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
418	`    elif category == "text":`	Testa uma condição alternativa quando as anteriores falharam.
419	`        result = convert_text(source, extension, output)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
420	`    else:`	Inicia o caminho alternativo quando a condição anterior for falsa.
421	`        result = convert_office(source, extension, output)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
422	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
423	`    if result.get("success"):`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
424	`        load_token(token, consume=True)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
425	`    return result`	Encerra a função e devolve o valor indicado ao chamador.
426	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
427	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
428	`def handle(message):`	Declara uma função Python reutilizável.
429	`    if not isinstance(message, dict):`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
430	`        return fail("Pedido inválido.")`	Encerra a função e devolve o valor indicado ao chamador.
431	`    action = message.get("action")`	Calcula ou armazena um valor que será usado pelas próximas linhas.
432	`    keys = set(message.keys())`	Calcula ou armazena um valor que será usado pelas próximas linhas.
433	`    if action == "pick-file" and keys == {"action"}:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
434	`        return pick_file()`	Encerra a função e devolve o valor indicado ao chamador.
435	`    if action == "convert-file" and keys == {"action", "selection", "format", "quality"}:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
436	`        return convert_file(message)`	Encerra a função e devolve o valor indicado ao chamador.
437	`    return fail("Pedido inválido.")`	Encerra a função e devolve o valor indicado ao chamador.
438	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.

439	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
440	`def main():`	Declara uma função Python reutilizável.
441	`    try:`	Inicia um bloco que pode gerar exceção.
442	`        message = read_message()`	Calcula ou armazena um valor que será usado pelas próximas linhas.
443	`        if message is None:`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
444	`            return`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
445	`        response = handle(message)`	Calcula ou armazena um valor que será usado pelas próximas linhas.
446	`    except Exception as error:`	Trata a exceção indicada sem encerrar abruptamente o host.
447	`        response = fail(f"Erro interno: {error}")`	Calcula ou armazena um valor que será usado pelas próximas linhas.
448	`    send_message(response)`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.
449	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
450	`(linha em branco)`	Linha em branco usada para separar visualmente blocos lógicos e melhorar a leitura.
451	`if __name__ == "__main__":`	Inicia uma condição; o bloco indentado executa quando ela for verdadeira.
452	`    main()`	Continuação sintática do bloco atual; completa a estrutura iniciada nas linhas anteriores.